



Automatizando tarefas com Triggers (Gatilhos)

Banco de Dados

Prof. Dr. Carlos Melo

 carlos.melo@upe.br



O Problema

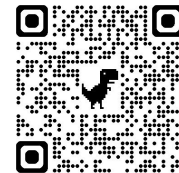
- Imagine um sistema onde:
 - Salários não podem ser negativos
 - Alterações precisam ser registradas
 - Algumas validações devem acontecer antes de alguma operação
- > E se o próprio banco pudesse reagir automaticamente para garantir cada um desses cenários?



Triggers (Gatilhos)

- Triggers são executadas automaticamente
- São respostas à eventos no banco
- Podem ocorrer antes ou depois de operações
- Eventos comuns:
 - **INSERT**
 - **UPDATE**
 - **DELETE**

Antes (**BEFORE**) e Depois (**AFTER**)



Tipo	Momento	Exemplo
BEFORE	antes da operação	validar salário
AFTER	depois da operação	registrar log

PROCEDURE VS TRIGGER



Procedure	Trigger
Chamada manualmente	Executa automaticamente
CALL	Evento da tabela
Ação Explícita	Reação Automática



Triggers (Gatilhos)

- > Nenhum Empregado pode Possuir um Salário Menor que um valor Mínimo
 - Primeiro criamos uma função que será chamada pelo Trigger

```
1 CREATE OR REPLACE FUNCTION validar_salario()  
2 RETURNS TRIGGER AS  
3 $$  
4 DECLARE  
5     salario_minimo NUMERIC;  
6 BEGIN  
7     salario_minimo := TG_ARGV[0];  
8     IF NEW.salario < salario_minimo THEN  
9         RAISE EXCEPTION 'Salário Inválido';  
10    END IF;  
11    RETURN NEW;  
12 END;  
13 $$  
14 LANGUAGE plpgsql;
```



Triggers (Gatilhos)

- > Nenhum Empregado pode Possuir um Salário Menor que um valor Mínimo
 - Então criamos o Trigger

```
1 CREATE TRIGGER trigger_validar_salario
2
3 BEFORE INSERT OR UPDATE
4 ON empregado
5
6 FOR EACH ROW
7
8 EXECUTE FUNCTION validar_salario(1621.0);
```



Triggers (Gatilhos)

```
1 CREATE TRIGGER trigger_validar_salario
2
3 BEFORE INSERT OR UPDATE
4 ON empregado
5
6 FOR EACH ROW
7
8 EXECUTE FUNCTION validar_salario();
```

Cria um novo gatilho

Antes da operação

Eventos que serão monitorados

Tabela que será observada

Executa em cada linha

Chama a função

> Toda a lógica fica na função e não no trigger



Triggers (Gatilho)

- Qual a saída esperada para a tentativa de atualização de salário abaixo?

```
1 UPDATE empregado
2 SET salario = 1100
3 WHERE id = "111.111.111-11";
```

Banco detecta o evento

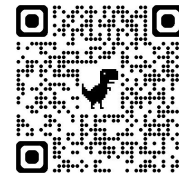
Trigger é acionada

Executa a função

Salário abaixo do mínimo

Erro gerado

OLD e NEW



Palavra	Significado	Exemplo
NEW	Novos valores da linha	Antes do UPDATE : OLD .salario = 3000
OLD	Valores antigos da linha	Depois do UPDATE : NEW .salario = 4000

> **OLD** e **NEW** permitem comparar alterações



Triggers para Auditoria

- Queremos um lugar para guardar o histórico de modificações
 - Uma tabela nova para log
- A tabela vai armazenar:
 - Quem foi alterado
 - Salário antigo
 - Salário novo
 - Data e hora da alteração



```
1 CREATE TABLE log_salario (  
2     id SERIAL PRIMARY KEY,  
3     cpf_empregado VARCHAR,  
4     salario_antigo NUMERIC,  
5     salario_novo NUMERIC,  
6     data_alteracao TIMESTAMP  
7 );
```



Triggers para Auditoria

- Novamente criamos uma nova função

```
1 CREATE FUNCTION registrar_log_salario()  
2 RETURNS TRIGGER AS  
3 $$  
4 BEGIN  
5  
6     INSERT INTO log_salario(  
7         cpf_empregado,  
8         salario_antigo,  
9         salario_novo,  
10        data_alteracao  
11    )  
12    VALUES(  
13        OLD.id,  
14        OLD.salario,  
15        NEW.salario,  
16        CURRENT_TIMESTAMP  
17    );  
18  
19    RETURN NEW;  
20  
21 END;  
22 $$ LANGUAGE plpgsql;
```

Salário antigo

Novo salário

Data e Hora atual



Triggers para Auditoria

- E só aí criamos o trigger

```
1 CREATE TRIGGER trigger_log_salario
2
3 AFTER UPDATE
4 ON empregado
5
6 FOR EACH ROW
7
8 EXECUTE FUNCTION registrar_log_salario();
```

Executa após a atualização

Para cada linha de empregado



Triggers para Auditoria

- Testando a Trigger

```
1 UPDATE empregado
2 SET salario = 4500
3 WHERE id = "111.111.111-11";
```

> Agora basta um **SELECT** maroto na tabela log_salario

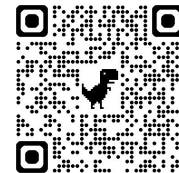


Outros usos de Triggers

- Validar dados
- Registrar alterações
- Impedir operações inválidas
- Atualizar informações automaticamente
- Implementar regras de negócio

Situação	Tipo
Impedir salário negativo	BEFORE
Registrar log	AFTER
Atualizar data automaticamente	BEFORE
Impedir exclusão	BEFORE DELETE

Exercícios



- Crie uma **TRIGGER** que impeça a remoção de departamentos



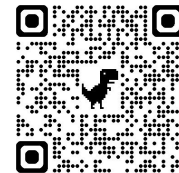
```
1 CREATE FUNCTION impedir_delete_departamento()  
2 RETURNS TRIGGER AS  
3 $$  
4 BEGIN  
5  
6     RAISE EXCEPTION  
7     'Departamentos não podem ser removidos!';  
8  
9 END;  
10 $$ LANGUAGE plpgsql;
```



```
1 CREATE TRIGGER trigger_delete_departamento  
2  
3 BEFORE DELETE  
4 ON departamento  
5  
6 FOR EACH ROW  
7  
8 EXECUTE FUNCTION impedir_delete_departamento();
```



```
1 DELETE FROM departamento  
2 WHERE numero = 1;
```

Exercícios

- Crie uma **TRIGGER** que impeça um empregado de participar de mais de 3 projetos

```
1 CREATE FUNCTION limitar_projetos()
2 RETURNS TRIGGER AS
3 $$
4 DECLARE
5     total INTEGER;
6 BEGIN
7
8     SELECT COUNT(*)
9     INTO total
10    FROM empregado_projeto
11    WHERE cpf_empregado = NEW.cpf_empregado;
12
13    IF total >= 3 THEN
14        RAISE EXCEPTION
15        'Empregado não pode participar de mais de 3 projetos!';
16    END IF;
17
18    RETURN NEW;
19
20 END;
21 $$ LANGUAGE plpgsql;
```

```
1 CREATE TRIGGER trigger_limitar_projetos
2
3 BEFORE INSERT
4 ON empregado_projeto
5
6 FOR EACH ROW
7
8 EXECUTE FUNCTION limitar_projetos();
```



Exercícios

- Crie uma **TRIGGER** que impeça reduções salariais acima de 30%
- Caso isso aconteça, gere uma exceção
- O que você vai precisar:
 - **OLD.salario**
 - **NEW.salario**
 - comparação entre valores (**IF**)
 - **BEFORE UPDATE**



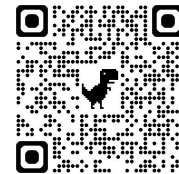
Exercícios

- Crie uma **TRIGGER** que registre automaticamente novos empregados em uma tabela chamada log_empregado
- O que você vai precisar:
 - cpf do empregado, nome, data/hora do evento
 - **AFTER INSERT**



Exercícios

- Crie uma **TRIGGER** que impeça a remoção de empregados vinculados a projetos
- O que você vai precisar:
 - **BEFORE DELETE**



Prof. Dr. Carlos Melo
<https://calendly.com/prof-casm>

 carlos.melo@upe.br